

Internship Report

(Day 4)

Report Type	Internship Daily Report
Day	Day 4
Name	Ibrahim Hasan
Date	20th May 2026
Task	Build a small Employee Management System Database with 4 tables. Then run SQL queries and document results.

Summary of Work

Step 1 — Create Database & Tables

The `employee_management` database was created and activated using the `CREATE DATABASE` and `USE` commands via the MySQL Shell and phpMyAdmin SQL interface. Four tables were then defined – `departments`, `employees`, `projects`, and `employee_projects` – each structured with appropriate data types, primary keys, and foreign key references to enforce relational integrity.

```
CREATE DATABASE employee_management;  
USE employee_management;
```

```
1 sql> CREATE DATABASE employee_management;  
2     USE employee_management;
```

Figure 1: CREATE DATABASE and USE statements executed in MYSQL Shell.

`departments` table:

Column	Type	Constraint	Null	Notes
department_id	INT	PK, AUTO_INCREMENT	No	Auto PK
department_name	VARCHAR(100)	NOT NULL	No	Required
location	VARCHAR(100)	–	Yes	Optional

```
1 sql> CREATE TABLE departments (  
2     department_id INT PRIMARY KEY AUTO_INCREMENT,  
3     department_name VARCHAR(100) NOT NULL,  
4     location VARCHAR(100)  
5 );
```

Figure 2: CREATE TABLE departments in MYSQL Shell.

`employees` table:

Column	Type	Constraint	Null	Notes
employee_id	INT	PK, AUTO_INCREMENT	No	Auto PK
employee_name	VARCHAR(100)	NOT NULL	No	Required
email	VARCHAR(100)	UNIQUE	Yes	Unique
phone	VARCHAR(20)	–	Yes	Optional
salary	DECIMAL(10,2)	NOT NULL	No	Required
joining_date	DATE	NOT NULL	No	Required
department_id	INT	FK → departments	Yes	FK ref
status	TINYINT(1)	DEFAULT 1	Yes	1=Active

```

1  sql> CREATE TABLE employees (
2      employee_id INT PRIMARY KEY AUTO_INCREMENT,
3      employee_name VARCHAR(100) NOT NULL,
4      email VARCHAR(100) UNIQUE,
5      phone VARCHAR(20),
6      salary DECIMAL(10,2) NOT NULL,
7      joining_date DATE NOT NULL,
8      department_id INT,
9      status BOOLEAN DEFAULT 1,
10     FOREIGN KEY (department_id) REFERENCES departments(department_id)
11 );

```

Figure 3: CREATE TABLE employees with SQL Shell.

projects table:

Column	Type	Constraint	Null	Notes
project_id	INT	PK, AUTO_INCREMENT	No	Auto PK
project_name	VARCHAR(100)	NOT NULL	No	Required
start_date	DATE	—	Yes	Optional
end_date	DATE	—	Yes	Optional
budget	DECIMAL(12,2)	—	Yes	Optional

```

1  sql> CREATE TABLE projects (
2      project_id INT PRIMARY KEY AUTO_INCREMENT,
3      project_name VARCHAR(100) NOT NULL,
4      start_date DATE,
5      end_date DATE,
6      budget DECIMAL(12,2)
7  );

```

Figure 4: CREATE TABLE projects with MYSQL Shell.

employee_projects table:

Column	Type	Constraint	Null	Notes
id	INT	PK, AUTO_INCREMENT	No	Auto PK
employee_id	INT	FK → employees	Yes	FK ref
project_id	INT	FK → projects	Yes	FK ref
assigned_date	DATE	—	Yes	Optional

```

1 sql> CREATE TABLE employee_projects (
2     id INT PRIMARY KEY AUTO_INCREMENT,
3     employee_id INT,
4     project_id INT,
5     assigned_date DATE,
6     FOREIGN KEY (employee_id) REFERENCES employees(employee_id),
7     FOREIGN KEY (project_id) REFERENCES projects(project_id)
8 );

```

Figure 5: CREATE TABLE employee projects with MYSQL Shell.

Step 2 — Insert Sample Data

Bulk INSERT INTO statements were executed to populate each table with 25 rows of realistic sample data. The `departments` table was seeded with Bangladeshi city locations; the `employees` table with local names, unique emails, and salary figures; the `projects` table with real-world project titles and budgets; and the `employee_projects` junction table with assignment records linking employees to projects.

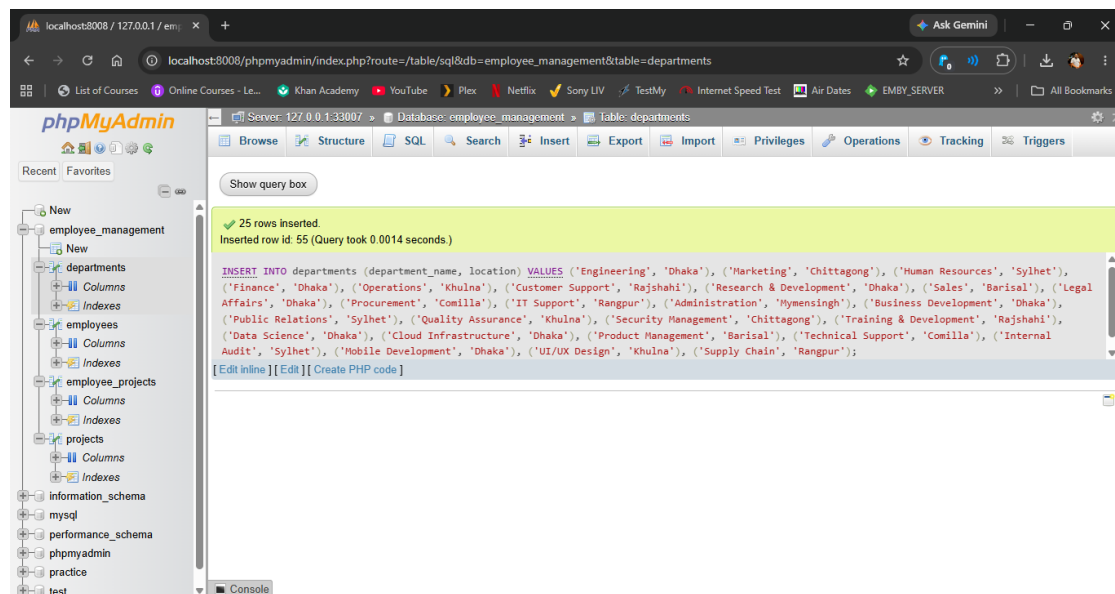


Figure 6: 25 rows inserted into ‘departments’ on phpMyAdmin confirmation

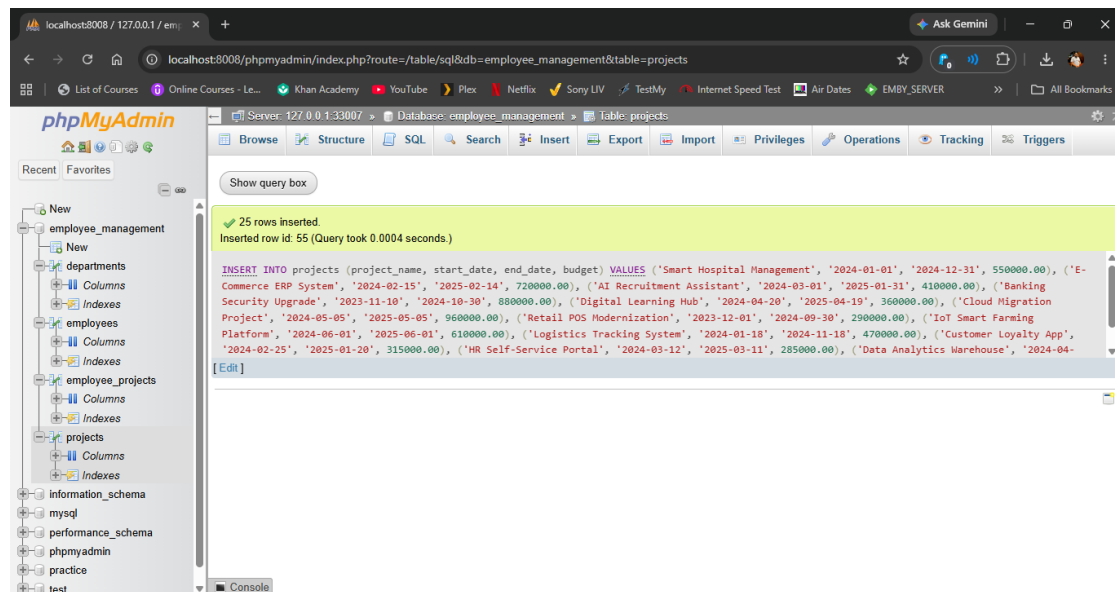


Figure 7: 25 rows inserted into 'projects' on phpMyAdmin confirmation.

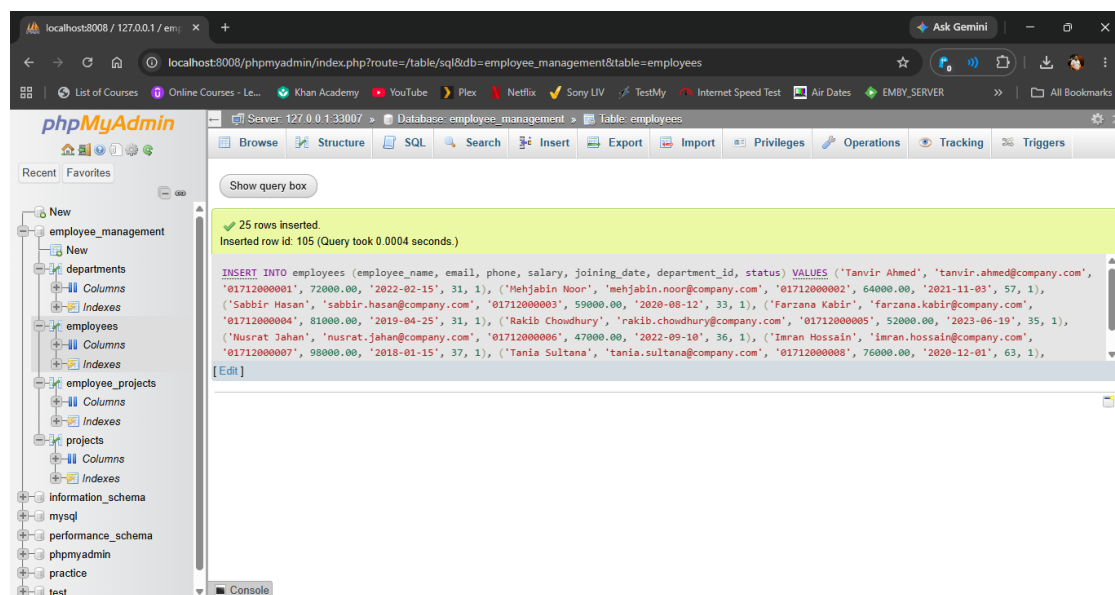


Figure 8: 25 rows inserted into 'employees' on phpMyAdmin confirmation.

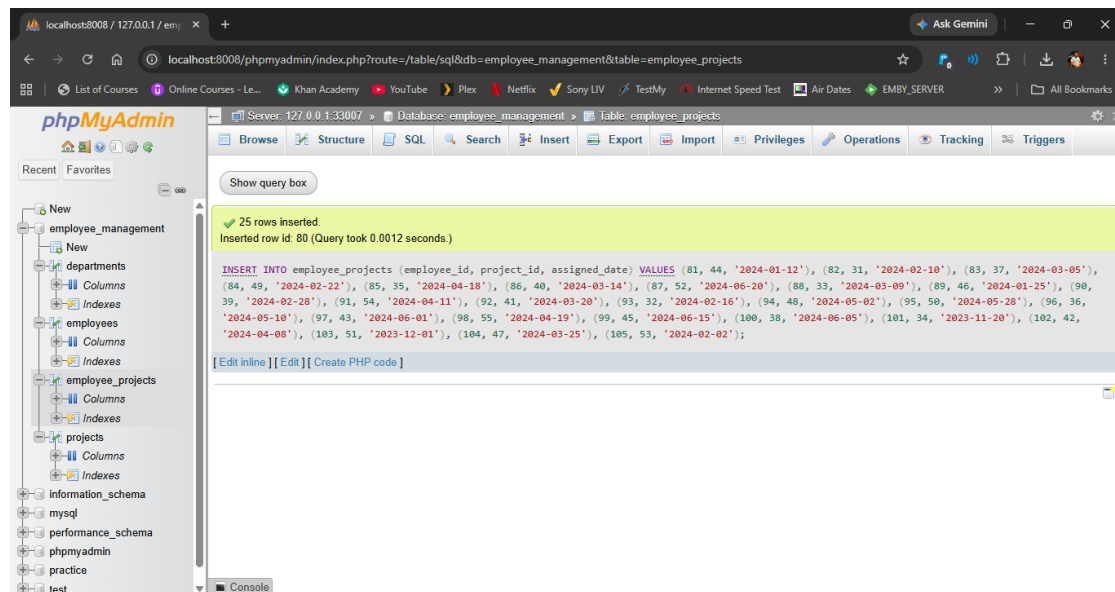


Figure 9: 25 rows inserted into 'employee projects' on phpMyAdmin confirmation.

Step 3 — Run SQL Queries

A series of SQL queries were executed against the populated database to retrieve and analyse data across multiple tables. Each query is documented below with its purpose and result.

Query 1 — Show all employees

```
SELECT * FROM employees;
```

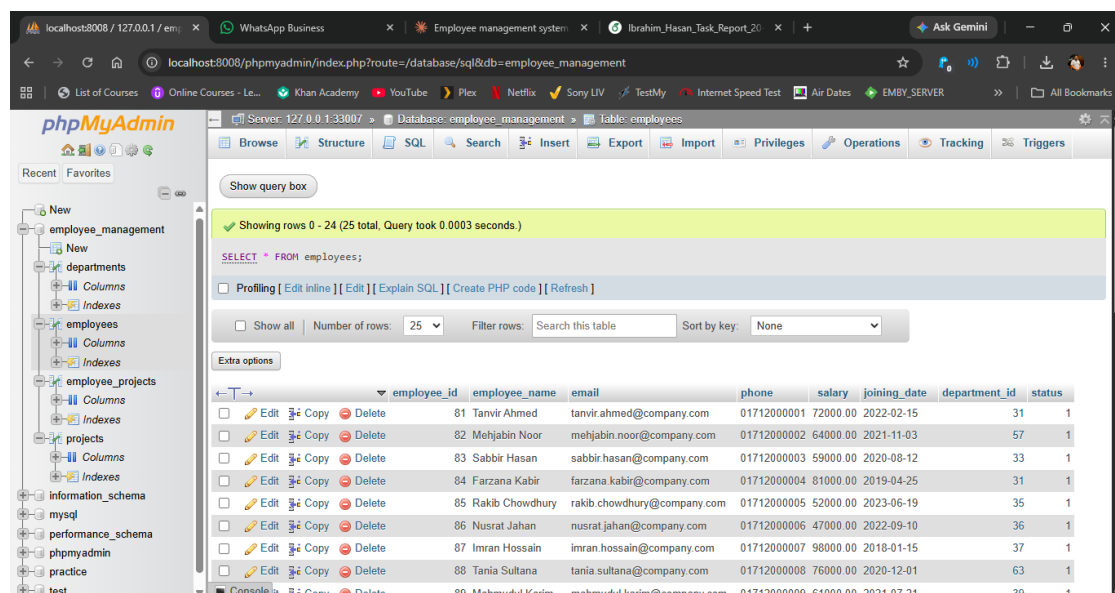
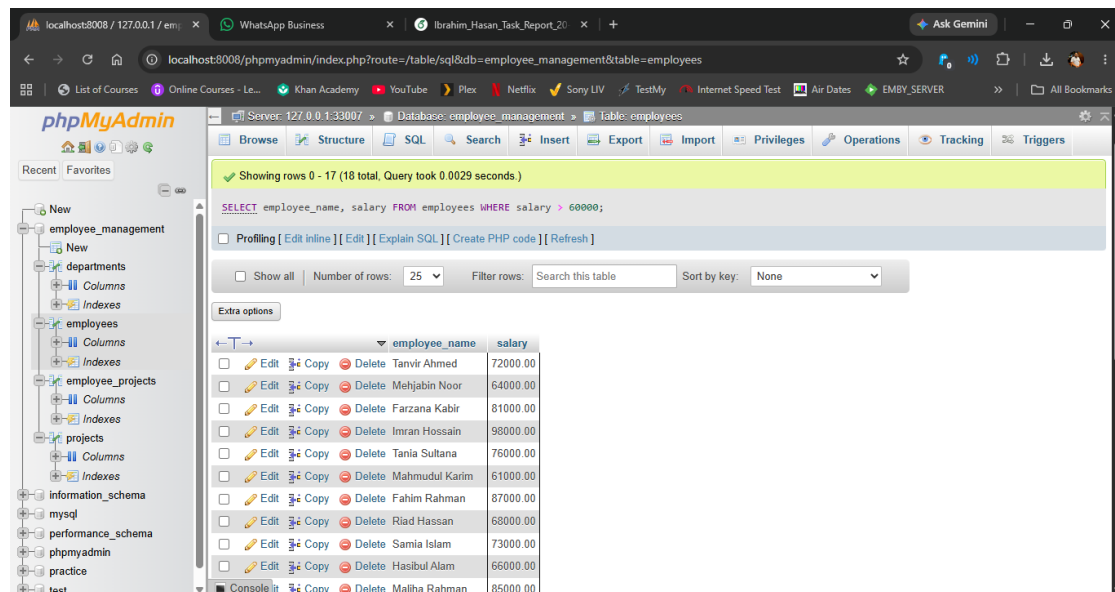


Figure 10: Query 1 result output in phpMyAdmin..

Query 2 — Show employees with salary above 60000

```
SELECT employee_name, salary FROM employees WHERE salary > 60000;
```

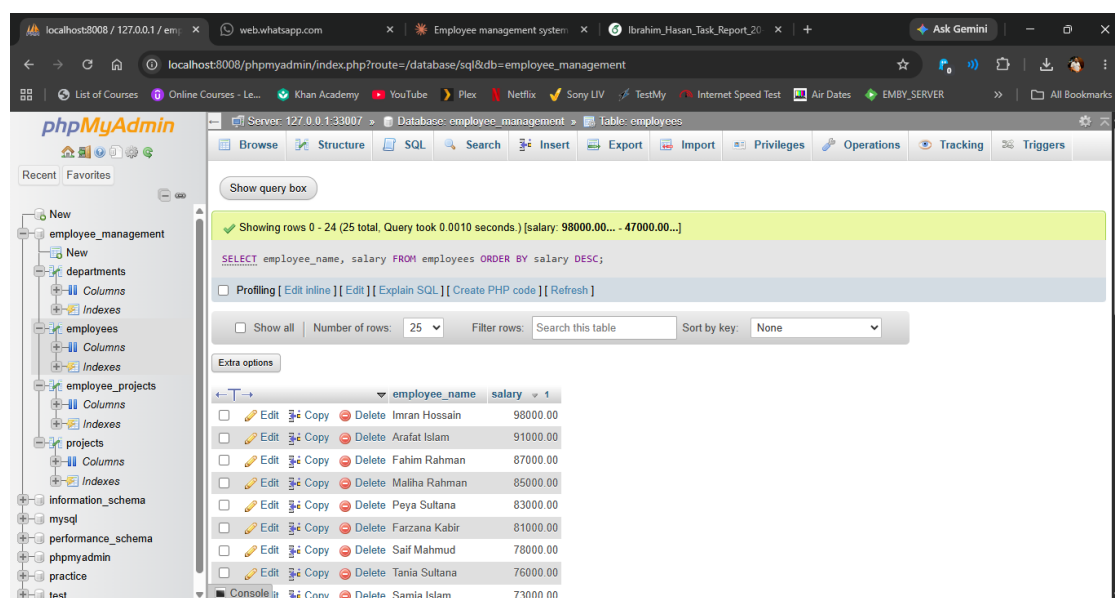


employee_name	salary
Tanvir Ahmed	72000.00
Mehjabin Noor	64000.00
Farzana Kabir	81000.00
Imran Hossain	98000.00
Tania Sultana	76000.00
Mahmudul Karim	61000.00
Fahim Rahman	87000.00
Riad Hassan	68000.00
Samia Islam	73000.00
Hasibul Alam	66000.00
Malha Rahman	85000.00

Figure 11: Query 2 result output in phpMyAdmin.

Query 3 — Show employees ordered by salary (highest first)

```
SELECT employee_name, salary FROM employees ORDER BY salary DESC;
```



employee_name	salary
Imran Hossain	98000.00
Arafat Islam	91000.00
Fahim Rahman	87000.00
Malha Rahman	85000.00
Peya Sultana	83000.00
Farzana Kabir	81000.00
Saif Mahmud	78000.00
Tania Sultana	76000.00
Samia Islam	73000.00

Figure 12: Query 3 result output in phpMyAdmin.

Step 4 — Verify Results & Document

All query outputs were reviewed to confirm accuracy and consistency with the inserted data. The database structure was verified through phpMyAdmin's Relation View, confirming that all foreign key constraints between the four tables are correctly established and operational.

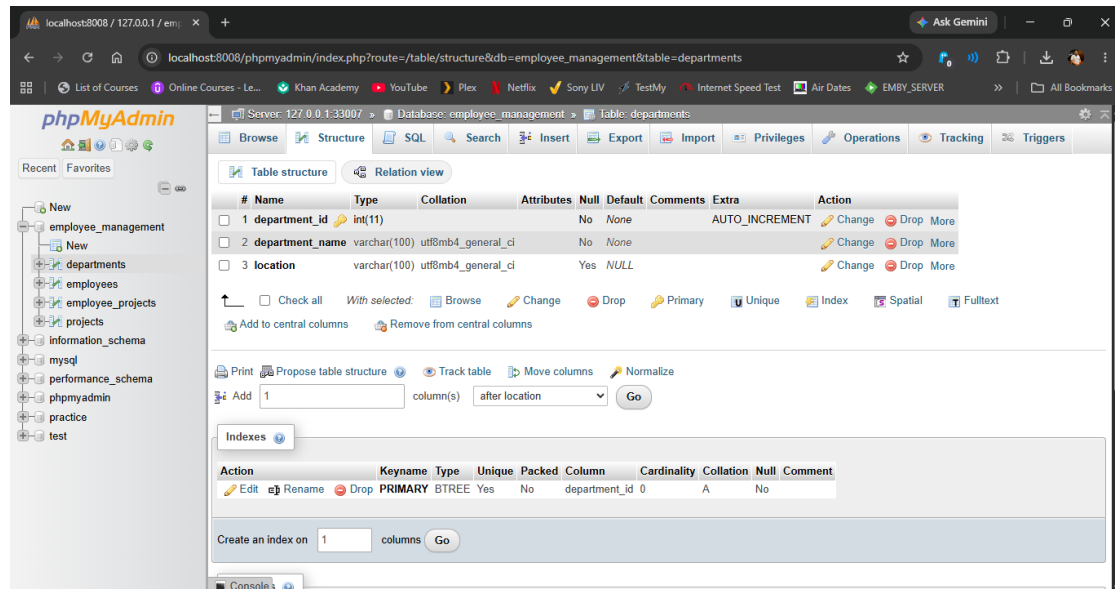


Figure 13: phpMyAdmin structure departments table.

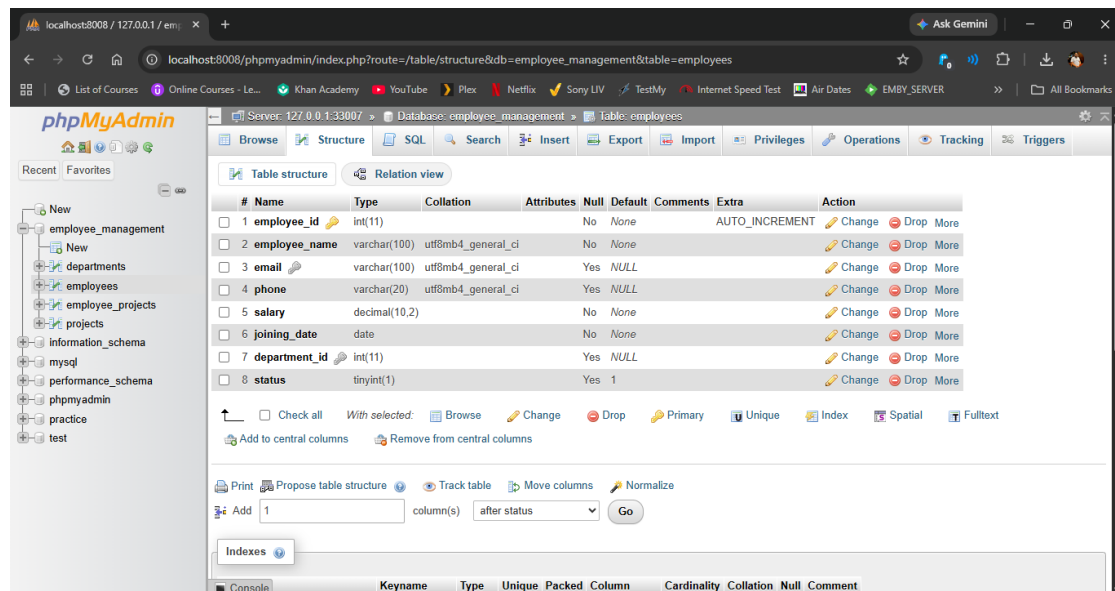


Figure 14: phpMyAdmin structure employees table.

Figure 15 shows the phpMyAdmin interface for the 'employee_projects' table. The table structure is as follows:

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	id	int(11)			No	None		AUTO_INCREMENT	Change Drop More
2	employee_id	int(11)			Yes	NULL			Change Drop More
3	project_id	int(11)			Yes	NULL			Change Drop More
4	assigned_date	date			Yes	NULL			Change Drop More

The 'Indexes' section shows the following:

Action	Keyname	Type	Unique	Packed	Column	Cardinality	Collation	Null	Comment
Edit Rename Drop	PRIMARY	BTREE	Yes	No	id	0	A	No	
Edit Rename Drop	employee_id	BTREE	No	No	employee_id	0	A	Yes	
Edit Rename Drop	project_id	BTREE	No	No	project_id	0	A	Yes	

Figure 15: phpMyAdmin structure employee projects table.

Figure 16 shows the phpMyAdmin interface for the 'projects' table. The table structure is as follows:

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	project_id	int(11)			No	None		AUTO_INCREMENT	Change Drop More
2	project_name	varchar(100)	utf8mb4_general_ci		No	None			Change Drop More
3	start_date	date			Yes	NULL			Change Drop More
4	end_date	date			Yes	NULL			Change Drop More
5	budget	decimal(12,2)			Yes	NULL			Change Drop More

The 'Indexes' section shows the following:

Action	Keyname	Type	Unique	Packed	Column	Cardinality	Collation	Null	Comment
Edit Rename Drop	PRIMARY	BTREE	Yes	No	project_id	25	A	No	

Figure 16: phpMyAdmin structure projects table.

Project Outcome

The `employee_management` database was successfully designed, implemented, and validated. All four tables were created with correct schema definitions, populated with 25 rows each (100 total records), and verified through SQL queries without error. Foreign key relationships between `departments`, `employees`, `projects`, and `employee_projects` are fully operational and consistent with the intended relational design. The environment is confirmed stable and ready for further development.

Step 5 — Run UPDATE and DELETE Queries

A series of UPDATE and DELETE queries were executed to test data manipulation and verify foreign key constraint compliance. The queries and their results are documented below.

Query 4 — Update employee salary

To test updates, we increased the salary of employee **Tanvir Ahmed** from **72000.00** to **75000.00**.

```
UPDATE employees SET salary = 75000.00 WHERE employee_name = 'Tanvir Ahmed';
```

```
Query OK, 1 row affected.  
Verified: Tanvir Ahmed's salary updated from 72000.00 to 75000.00.
```

Query 5 — Delete employee record

To test deletions and cascading constraints, we removed employee **Mustafizur Rahman**. The corresponding assignments in the employee_projects junction table should be deleted automatically due to the ON DELETE CASCADE constraint.

```
DELETE FROM employees WHERE employee_name = 'Mustafizur Rahman';
```

```
Query OK, 1 row affected.  
Cascade Verification: Assignments for Mustafizur Rahman in employee_projects  
dropped from 1 to 0.  
Verified: Record for Mustafizur Rahman has been deleted successfully.
```